In [ ]:
```
%pip install pdfplumber pandas numpy scikit-learn
```

In [1]:
```
import re
import pdfplumber
import pandas as pd
import numpy as np
from pathlib import Path

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

In [2]:
```
pdf_path = Path("heart disease.pdf")

if not pdf_path.exists():
    raise FileNotFoundError(
        "Please place 'heart disease data.pdf' "
        "in the same folder as this notebook."
    )

text_pages = []

with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
        text_pages.append(page.extract_text() or "")

all_text = "\n".join(text_pages)

print("PDF loaded successfully.")
print("Pages:", len(text_pages))
```

```
PDF loaded successfully.
Pages: 12
```

In [3]:
```
columns = [
    "age", "sex", "cp", "trestbps", "chol", "fbs",
    "restecg", "thalach", "exang", "oldpeak",
    "slope", "ca", "thal", "num"
]

rows = []

for line in all_text.splitlines():
    line = line.strip()
    parts = [part.strip() for part in line.split(",")]

    if len(parts) == 14 and all(
        re.fullmatch(r"\?|[-+]?\d+(?:\.\d+)?", x)
        for x in parts
    ):
        rows.append(parts)

if not rows:
    raise ValueError(
        "No data rows were extracted. Check the PDF format."
```

```
    )

df = pd.DataFrame(rows, columns=columns)
df = df.replace("?", np.nan)

for col in columns:
    df[col] = pd.to_numeric(df[col], errors="coerce")

print("Rows extracted:", len(df))
print("Columns:", len(df.columns))

display(df.head())
```

```
Rows extracted: 1025
Columns: 14
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	nu
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	

In [4]:

```
print("Missing values before cleaning:")
display(df.isnull().sum())

for col in df.columns:
    df[col] = df[col].fillna(df[col].median())

df["target"] = (df["num"] > 0).astype(int)

print("Dataset shape:", df.shape)
print("Target counts:")

display(
    df["target"].value_counts().rename(
        index={0: "No disease", 1: "Disease present"}
    )
)
```

```
Missing values before cleaning:
```

```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
num         0
dtype: int64
Dataset shape: (1025, 15)
Target counts:
target
Disease present    526
No disease         499
Name: count, dtype: int64
```

In [5]:

```
model_df = df[
    ["age", "sex", "cp", "trestbps", "chol", "thalach", "target"]
].copy()

model_df["age_group"] = pd.cut(
    model_df["age"],
    bins=[0, 40, 55, 120],
    labels=["young", "middle", "senior"],
    include_lowest=True
).astype(str)

model_df["bp_group"] = pd.cut(
    model_df["trestbps"],
    bins=[0, 120, 140, 300],
    labels=["normal", "elevated", "high"],
    include_lowest=True
).astype(str)

model_df["chol_group"] = pd.cut(
    model_df["chol"],
    bins=[0, 200, 240, 700],
    labels=["desirable", "borderline", "high"],
    include_lowest=True
).astype(str)

model_df["thalach_group"] = pd.cut(
    model_df["thalach"],
    bins=[0, 100, 150, 300],
    labels=["low", "medium", "high"],
    include_lowest=True
).astype(str)

model_df["cp_group"] = model_df["cp"].astype(int).astype(str)
model_df["sex_group"] = model_df["sex"].astype(int).astype(str)
```

```
bn_data = model_df[
    [
        "age_group", "sex_group", "cp_group",
        "bp_group", "chol_group", "thalach_group", "target"
    ]
].copy()

display(bn_data.head())
```

	age_group	sex_group	cp_group	bp_group	chol_group	thalach_group	target
0	middle	1	0	elevated	borderline	high	0
1	middle	1	0	elevated	borderline	high	0
2	senior	1	0	high	desirable	medium	0
3	senior	1	0	high	borderline	high	0
4	senior	0	0	elevated	high	medium	0

In [6]:
```
features = [
    "age_group", "sex_group", "cp_group",
    "bp_group", "chol_group", "thalach_group"
]

prior = bn_data["target"].value_counts(
    normalize=True
).sort_index()

print("Prior probability of target:")
display(prior)

cpts = {}

for feature in features:
    cpts[feature] = pd.crosstab(
        bn_data[feature],
        bn_data["target"],
        normalize="columns"
    ).fillna(0)

print("Conditional probability table for chest-pain category:")
display(cpts["cp_group"])
```

```
Prior probability of target:
target
0    0.486829
1    0.513171
Name: proportion, dtype: float64
Conditional probability table for chest-pain category:
```

target	0	1
cp_group		
0	0.751503	0.231939
1	0.066132	0.254753
2	0.130261	0.416350
3	0.052104	0.096958

In [7]:
```
def conditional_probability(feature, value, target_value):
    subset = bn_data[
        bn_data["target"] == target_value
    ][feature]

    categories = bn_data[feature].nunique()
    count = (subset == str(value)).sum()

    return (count + 1) / (len(subset) + categories)


def predict_disease(sample):
    scores = {}

    for target_value in [0, 1]:
        prior_prob = (
            bn_data["target"].eq(target_value).sum() + 1
        ) / (len(bn_data) + 2)

        score = prior_prob

        for feature in features:
            score *= conditional_probability(
                feature,
                sample[feature],
                target_value
            )

        scores[target_value] = score

    total = sum(scores.values())

    probabilities = {
        key: value / total
        for key, value in scores.items()
    }

    prediction = max(probabilities, key=probabilities.get)

    return prediction, probabilities
```

In [8]:
```
sample_patient = {
    "age_group": "senior",
```

```
    "sex_group": "1",
    "cp_group": "4",
    "bp_group": "elevated",
    "chol_group": "high",
    "thalach_group": "medium"
}

prediction, probabilities = predict_disease(sample_patient)

print("Example input:", sample_patient)

print(
    "Predicted class:",
    "Disease present" if prediction == 1 else "No disease"
)

print("Estimated probabilities:", probabilities)
```

```
Example input: {'age_group': 'senior', 'sex_group': '1', 'cp_group': '4', 'bp_grou
p': 'elevated', 'chol_group': 'high', 'thalach_group': 'medium'}
Predicted class: No disease
Estimated probabilities: {0: np.float64(0.8814096619665689), 1: np.float64(0.1185903
3803343119)}
```

In [9]:
```
print("Practical 3 completed.")
print("Dataset records used:", len(bn_data))
print("Input nodes:", ", ".join(features))
print("Output node: target (0 = no disease, 1 = disease present)")
print("Reminder: This model is only for academic demonstration.")
```

```
Practical 3 completed.
Dataset records used: 1025
Input nodes: age_group, sex_group, cp_group, bp_group, chol_group, thalach_group
Output node: target (0 = no disease, 1 = disease present)
Reminder: This model is only for academic demonstration.
```